

sectors. We can also express size in units of KiB, MiB or GiB.

Once the partition boundary is entered, *fdisk* will display a confirmation that the partition has been created.

Define the partition type

If the newly created partition should have a type other than Linux, enter the *t* command to change the type of the partition. Enter the hex code for the new partition type. If needed, a table of hex codes for all partition types can be displayed with the *L* command. Setting this partition type correctly is crucial, since some tools rely on it to function properly.

Save the partition table changes

Issue the *w* command to finalise the partition-creation request by writing the changes to the disk partitions table and exiting the *fdisk* program.

Initiate a kernel read of the new partition table

Run the *partprobe* command with the disk device's name as an argument to force a re-read of its partition table.

Creating a file system

After a block device has been created, the next step is to apply a file system format to it. A file system applies a structure to the block device so that data can be stored and retrieved from it. Red Hat Linux supports many file systems, two of the most common being *xfs* and *ext4*.

The *mkfs* command can be used to apply a file system to a block device. If no type is specified and extended, the type two (*ext2*) file system will be used, which is not desirable in many cases. To specify the file system type, the command -*t* should be used.



Figure 1: The creation of a partition of 300MiB



Figure 2: File system of the partition has been created

```
# mkfs -t ext4 /dev/vdb1
```

Mounting file systems

Once the file system format has been applied, the last step to adding a new file system is to attach a file system into a new directory structure. When the file system is attached into the directory's hierarchy, user space utilities can access or write files on the device.

Manually mounting file systems

Systems administrators can use the *mount* command to manually attach the device into the directory's location, or mount point, by specifying the device and the mount point, as well as any options that may be desired, to customise the behaviour of the device. We can see the mounted file system by using the command *df -h*.

```
# mount /dev/vdb1 /share
```

Directory (any name) should be created manually before running the *mount* command. The name of the created directory will be specified with *mount* command. In Figure 3, a directory with the name/share has been created. The *mount* command can also be used to view currently mounted file systems, the mount point and the options.

Manually mounting a file system is an excellent way to verify that a formatted device is accessible or working in the way desired. However, once the system is rebooted, the file system (while it still exists and has intact data) will not be mounted into the directory tree again. If an administrator wants the file system to be permanently mounted, a listing of the file system needs to be added to */etc/fstab*.

Persistently mounting file systems

By adding a listing for a device into the */etc/fstab* file, the administrator can configure a device to be mounted to a mount point at system boot.

/etc/fstab has six fields per line. The first field specifies the device to be used. The *UUID* can be used to specify the device or, alternatively, the device file could be used. The *UUID* is stored in the file system superblock and created when the file system is created.

The second field is the mount point where the device should be attached into the directory's hierarchy. The mount point should already exist, and if it does not, it can be created by using the *mkdir* command.

The third field contains the file system's type that has been



Figure 3: The file system has been mounted to the /share directory